

אוניברסיטת בן גוריון
בית הספר להנדסת חשמל ומחשבים

דוח תיעוד פרויקט גמר קורס "מבנה מחשבים ספרתיים"
361-1-4191

Kobi's and Or's Project

מגישים: אור יעקבי 206827164
יעקב קוזמינסקי 206511966

מדריך אחראי : מר חנן ריבוא

22.8.2024

הגדרת ומטרת הפרויקט:

תכנון ומימוש מערכת משובצת מחשב מבוססת MCU לצורך בקרת מכונה מבוססת מנוע צעד בשליטה ידנית ע"י analog joystick בנוסף לשליטה מרחוק ממחשב אישי דרך ערוץ תקשורת טורית. הדגשים בתכנון המערכת הם של רמת ביצועים גבוהה, הספק נמוך ככל שניתן תחת משטר hard real time גבוה.

תיאור הפרויקט:

הפרויקט בנוי משתי מערכות המתקשרות ביניהן בערוץ תקשורת. מערכת אחת היא הבקר msp430 של חברת Texas Instruments המערכת השנייה היא ה-PC.

כניסות המערכת – בצד בקר, joystick אנלוגי הבנוי מלחצן וטווח מתחים בשני צירים 0-3.3V המצביעים על מיקומו של joystick בצד PC, לחצני המקלדת 1,2,3,4, space, Enter, כאשר 1-4 משמים למעבר בין המשימות בתפריט הראשי.

Enter – התקדמות ביצוע תכנית ב script mode (מצב 4)

Space – אתחול מערכת + מעבר לתפריט הראשי.

יציאות המערכת – בצד בקר, LCD ו Stepper Engine אותם.

בצד PC, מערכת GUI המאפשרת ציור וממשק נוח למשתמש לעבור בין מצבים ולדעת מה הם התהליכים המתבצעים במערכת בזמן אמת.

פירוט המצבים (פעולות שהמערכת יכולה לבצע) –

מצב 1 – המשתמש מזיז את Stepper Engine ע"י joystick

מצב 2 – המשתמש מצייר על גביי ה-PC ע"י joystick

מצב 3 – כיול Stepper Engine

מצב 4 -- Script Mo בו המשתמש יכול לעלות קובץ text למערכת המכיל פקודות מתוך רשימה של 8 פקודות ולתת למערכת לבצע את הפקודות בזו אחר זו ע"י לחיצה על מקש Enter.

רשימת פקודות High Level נדרשות לתמיכה במצב של Script Mode:

OPC (first Byte)	Instruction	Operand (next Bytes)	Explanation
0x01	inc_lcd	x	Count up from zero to x with delay d onto LCD
0x02	dec_lcd	x	Count down from x to zero with delay d onto LCD
0x03	rra_lcd	x	Rotate right onto LCD from pixel index 0 to pixel index 31 a single char x (ASCII value) with delay d
0x04	set_delay	d	Set the delay d value (units of 10ms)
0x05	clear_all_leds		Clear LCD
0x06	stepper_deg	p	Points the stepper motor pointer to degree p and show the degree (dynamically) onto PC screen
0x07	stepper_scan	l,r	Scan area between left l angle to right r angle (once) and show the start and final degrees (right on time the motor pointer reaches them) onto LCD screen
0x08	sleep		Set the MCU into sleep mode

חלוקת עבודה בין PC לmsp430

הקדמה:

כפי שנאמר קודם, הפרויקט בנוי מ-2 מערכות וכעת נפרט על תפקידם של כל אחת מהן במשימות השונות וכיצד ביצענו לפי סדר בניית הפרויקט בפועל.

המערכת מתחילה במצב שינה, כאשר כל הפורטים והפריפריות בהן נשתמש מקונפגות ומוכנות להפעלה. ערוץ הקבלה בצד בקר RX מוכן לקבל מידע מהPC. בצד PC בזמן זה בעזרת הGUI מראה את התפריט למשתמש ובלחיצת כפתור על אחד מהמספרים 1-4 נשלחת הבחירה לבקר דרך RX מעירה אותו ומכניסה אותו למצב הנבחר.

בצד PC השתמשנו בשפת python ובספריות Tkinter וPyserial

Tkinter - ממשק מול המשתמש בלולאה אינסופית הGUI עצמו
Pyserial – שליחת מידע באופן טורי וחיבור לצד בקר
Tkinter – מקבול פעולות השליחה עם הפעולות הגרפיות בצד מסך.

בצד בקר השתמשנו בשפת C וספריות Stdio וmath בלבד.
ב UART, GPIO, ADC, Basic Timer פירוט החיבורים:

P1.0 y-axis - Joystick , analog input to ADC
P1.1,P1.2 – UART
P1.3 x-axis Joystick , analog input to ADC
P1.4 - JOYSTICK_SW -Push button interrupt
P1.5-P1.7 – LCD Control
P2.0-P2.3- output to Stepper Engine (Phase A – D)
P2.4-P2.7 LCD Data

Mode 3 Calibration

במצב זה לא קיימת תקשורת בין הPC לבקר.
חיברנו את הדרייבר של המנוע צעד אל הבקר ודאגנו להוציא גלים ריבועיים בסדר נכון ובתדר מקסימלי בעזרת Basic Timer – כך שהמנוע יסתובב עם כיוון השעון.
על תחילת וסוף הסיבוב אנחנו משפיעים בעזרת פסיקה חומרתית של הjoystick- אותו חיברנו לפורט P1.4. בהינתן פסיקה הנובעת מלחיצה על הjoystick - עולה דגל בינארי המאפשר לפונקציה לפעול ולהזיז את המנוע צעד אחר צעד. בהינתן פסיקה שנייה- הדגל הנ"ל יורד ובכך נעצר סיבוב המנוע.
עם עצירת המנוע משתנה curr_location המצביע על מיקומו של מנוע הצעד מתאפס ונקודה זו מוגדרת להיות נקודת האפס החדשה של המנוע.
בכל צעד עם כיוון השעון משתנה זה עולה באחד ונגד כיוון השעון יורד באחד.
מבדיקה שלנו סיבוב שלם של המנוע הוא 510 צעדים בדיקה זו תעזור לחישובים שיפורטו בהמשך.

- Mode 1 Manual Control

הפונקציה השנייה אותה כתבנו הייתה פונקציית השליטה הידנית במנוע צעד. גם במצב זה אין תקשורת עם ה-PC לשם כך היה עלינו לקפנא את ה-ADC ולמדוד את המתחים בציר X ו Y אשר מצביעים על מקום המצאות הjoystick. מצב העבודה של ה-ADC הוא רציף ובמספר ערוצים בו זמנית על מנת לקבל עבור כל אחד מ-2 הסיגנלים X ו Y 4 דגימות בכל הפעלה. כך שכל עוד אנחנו ב מצב 1 ה-ADC דוגם את ערכי המתח מהjoystick ממצע את הדגימות לכל סיגנל על מנת לקבל רמת דיוק גובהה ומפעיל את המנוע הצעד בהתאם. החישוב מתבצע ע"י מדידה של מתח בציר X ו מתח בציר Y. ובעזרת ספריית MATH השתמשנו בפונקציית Atan2 שעושה בעצם ארק-טנגנס של Y/X והמרה למעלות. מכאן חישוב מספר הצעד אליו צריך להגיע הוא פשוט. $\text{Angle} * 510 / 360$. פרט לכך ישנם עוד אתגרים בחישוב איתם התמודדנו כגון: התאמת תחום המנוחה של המנוע לפי הג'ויסטיק. זוויות שליליות, נקודת ייחוס לזווית 0 שונה בין המישור המתמטי x-y לבין מנוע הצעד. מקרי קצה בהשלמת סיבוב/ הגעה ל-180 מעלות וכו'.

- Mode 2 Draw Mode

הפונקציה השלישית אותה כתבנו הייתה פונקציית הציור בעזרת הjoystick על גבי מסך המחשב. לשם כך גם כאן השתמשנו ב-ADC על מנת למדוד מתחים באופן רציף ב-2 הערוצים של הjoystick, גם כאן בכל איטרציה מתבצעות 4 דגימות עבור כל סיגנל ומיצוע לקבלת דיוק גבוה. ה-ADC מתאר ערכים בין 0-1023 כי מדובר ב-ADC10 ולו 10 ביט. מכיוון שבמוד יש זה תקשורת עם המחשב, נבצע את כל החישובים בו- כך התקשורת והפעלת המערכת תהיה מהירה יותר. לשם כך, אנו מפרקים את הערכים המדודים ל-4 ספרות- מאלפים ליחידות והופכים אותן לטיפוס char, מכיוון שהתקשורת היא תקשורת טורית. בנוסף, לסוף מערך השליחה נוסיף את התו "~", תו זה יהווה אינדוקציה למחשב שזוהי סוף של דגימה. וכעת הוא ידע לעצור לקבל מידע ולעבד אותו לכיוון התקדמות של הצייר.

בצד המחשב: יתבצע עיבוד המידע בצורה הבאה:

חילקו את מרחב מתחי הג'ויסטיק ל-9 חלקים- כאשר כל חלק מהווה כיון התקדמות אפשרי של הצייר. את החלקים תחמנו לפי מתחי סף של ה-ADC עם טווח ערכים מסוים לכל חלק, זאת על מנת לייצב רעש. חלוקה זו, היא חלוקה המתאימה ביותר מכיוון שהפיקסלים בתמונה מחולקים באותה צורה. אחרי חישוב כיון ההתקדמות, נעדכן את מיקום הצייר בכיוון המחושב בדלתא של 2 פיקסלים.

במצב זה ישנם שלושה מצבי ציור: ניוטרל, ציור ומחיקה. המעבר בין מצבים אלו ינבע מלחיצה על הג'ויסטיק. זאת תקפיץ פסיקה לבקר אשר בתורו ישנה את המצב הפנימי ובמערך שליחת התווים של דגימות המתחים של ה-ADC – התו הראשון- מהווה את מספר המצב.

אפיון התקשורת:

במצב זה מימשנו מערכת ack-req. כאשר המחשב מסיים לעבד את המידע, הוא מודיע לבקר שהוא מוכן לקבל עוד דגימה חדשה- ע"י שליחת תו ייעודי. בקבלת תו זה- הבקר יוצא ממצב שינה ומפעיל את הADC, דוגם ושולח את הדגימה הבאה. כעת, יופעל thread אשר ממקבל את פעולת הGUI ועדכון מיקום הצייר לבין המתנה לקבלת הדגימה בנוספת. כך אנו משפרים את עבודת המערכת, לא מעמיסים מידע על הPC, על ערוץ התקשורת, חוסכים אנרגיה בבקר והכי חשוב- שומרים על כך שהדגימות שמגיעות את המחשב הן דגימות בזמן אמת ולא עיבוד של דגימות עבר שהמחשב לא היה פנוי לעבד.

– Mode 4 Script Mode

במצב זה, נרצה לבנות מערכת אשר יודעת לתרגם קובץ טקסט שנטען במחשב לרצף פקודות שישמרו בבקר.

לשם כך, ראשית בנינו פונקצייה בצד הPC אשר יודעת להמיר קובץ טקסט לסט פקודות המכילות אופקוד ואופרנדים, לפי מילון פנימי. בסוף המרה של כל קובץ, הפונקציה מוסיפה תו נוסף 'z' המעיד על סוף קובץ.

הוספנו בGUI מסך שבו ישנה אפשרות של טעינה של עד 3 קבצים. כל קובץ שנטען עובר תרגום לקוד מכונה ובלחיצת כפתור מתאים על הGUI- הקבצים נשלחים לבקר. ומראים למשתמש הודעה כאשר הקבצים נשלחו והתקבלו בהצלחה בבקר.

כעת בצד הבקר, הגדרנו אובייקט חדש שמסוגל להכיל בתוכו 3 סטים של קוד מכונה. כאשר הבקר במוד 4 הוא מוכן לשמור ערכים לאובייקט הנ"ל. בהנחת התקבלת תו Z הבקר יודע שזהו סוף של קובץ. ולכן הוא עובר לשמירה של ערכים בקובץ הבא (אם ישנם).

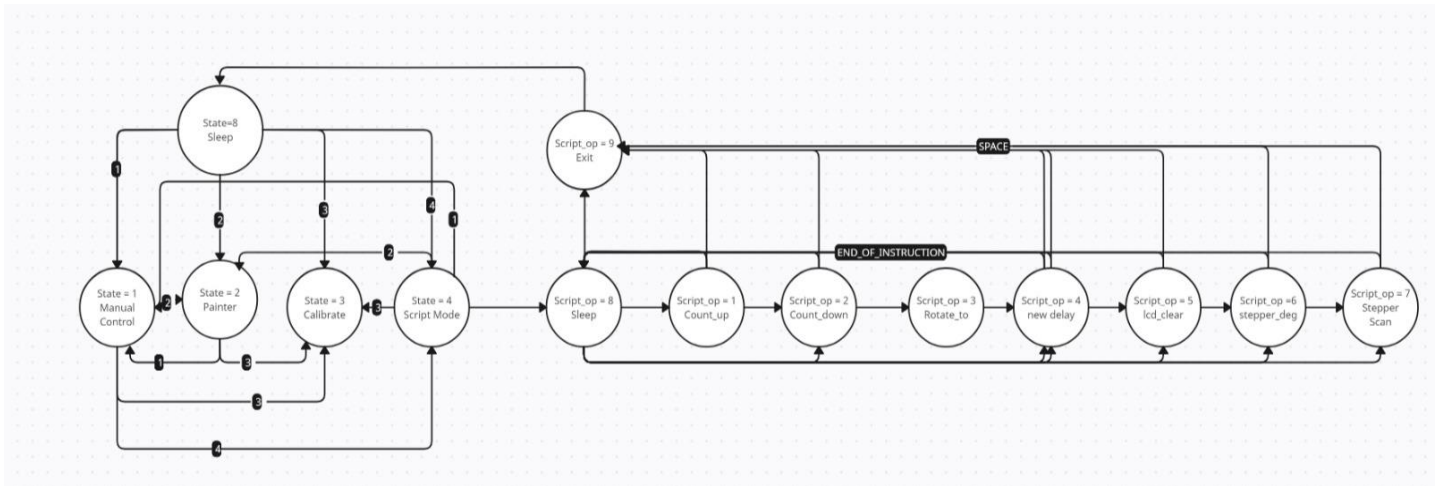
לאחר קבלת הקבצים וטעינתם בצד הבקר, למשתמש בPC עולה הודעה שמאפשרת לו להריץ את התוכנה שנצרבה. בלחיצת כפתור ENTER הבקר מקבל עוד תו ייעודי 'q' שאומר לו להתחיל ולקרוא את הפקודות שנשמרו אצלו. כך המתמש יכול לקרוא את כל הפקודות שנמצאות על הבקר.

רוב פקודות הסקריפט בנויות מפונקציות שנבנו במהלך הקורס במעבדות הקודמות, המשתמשות בפריפריות כגון שעון, LCD וכן פונקציות מורכבות יותר של המנוע צעד.

פקודות מיוחדות:

ישנה פקודה במצב זה אשר בה נדרש להציג באופן דינמי את הזווית של המנוע צעד על המסך. זאת נבצע באיטרציות, נחשב את כמות הצעדים שעל המנוע לבצע- לפי מיקום נוכחי ומיקום יעד. ואז לאחר כל צעד, נשלח לPC את המיקום הנוכחי. כעת נשתמש שוב בשיטת ack-req בה המחשב יאותת לבקר בסוף קבלה שהבקר יכול להתקדם צעד נוסף. בזמן שהבקר מתקדם- הPC פותח thread אשר מציג את הזווית הנוכחית על המסך ובמקביל פנוי לקבל זווית חדשה- זו עוד שיטה של יעול זמנים והצגה נכונה של נתונים בזמן אמת.

מערכת ה-FSM:

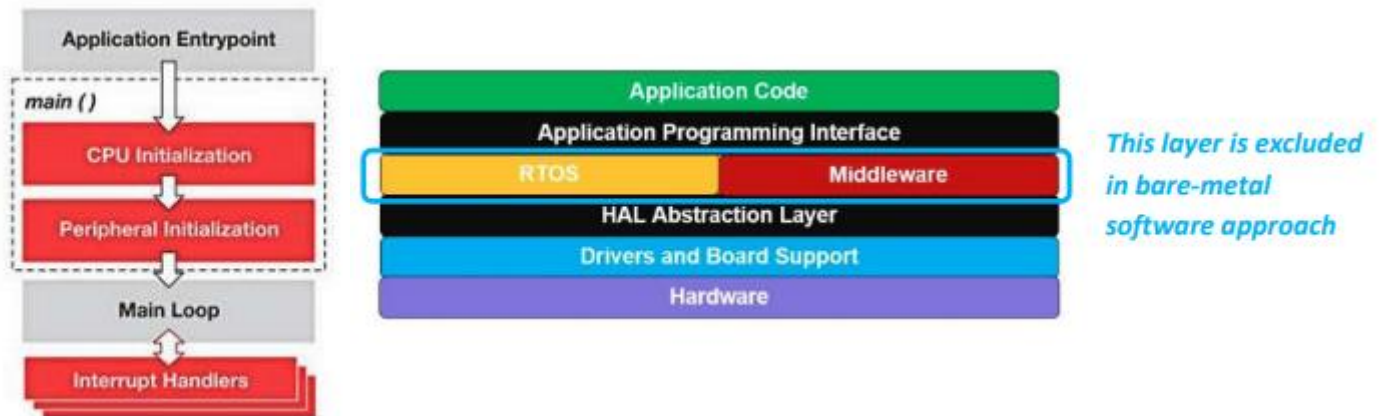


הערות:

- במצב 4, ברגע שנטען קובץ על המשתמש לחזור לתפריט הראשי על מנת להגיע למצבים אחרים במערכת.

זאת משום שעל המערכת לאפס את הזיכרון ולאתחל את המצביעים של הקבצים השונים. על מנת להשתמש בצב 4 או 1 חובה על המשתמש לבצע כיוול ע"י מצב 3

שכבות האבסטרקציה:



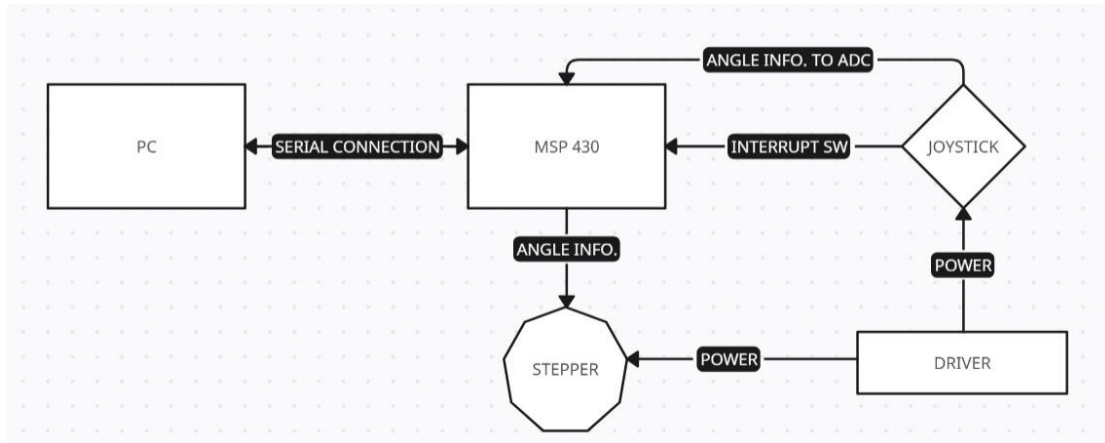
1. שכבת ה Hardware - מכילה קוד הקשור לקנפוג אופני עבודה של המעבד, מצב עבודה של שעון, MCLK, SMCK אתחול טבלאות vectors interrupt רמות העדיפות וכו' בפרויקט זה לא התעסקנו בשכבה זו.
2. שכבת BSP (Board Support Package) מכילה קוד לקנפוג רגיסטרים של הרכיבים הפריפריאליים השונים של הבקר, קנפוג הבסים, תדרי העבודה שלהם וכו.
3. שכבת ה HAL מכילה את כל הפונקציות וחלקי הקוד המפעילים את הרכיבים הפריפריאליים ורוטינות האינטראפט, וערוצי התקשורת.
4. שכבת ה API - מכילה את הקוד בו אנו כותבים את האפליקציה של המערכת ב Level High תוך גישה לרכיבים פריפריאליים דרך API בלבד כאשר המימוש של השכבות מטה "שקוף" לשכבה זו, קוד הוא portable כך שהוא יהיה תקף גם במידה וה MCU - של המערכת יתחלף באחר. בשכבה זו מתבצעים כל החישובים השייכים לצד בקר כמפורט למעלה, פונקציות תזמון של הפריפריות (מפעילות את הפריפריות בהתאם לצורכי שכבה ה API) פונקציות הסתעפות בהתאם לקבלת מידע משכבה ה HAL .
5. שכבת ב Application - היא כבר שכבת קוד הגבוהה ביותר בה מתקיים הממשק עם המשתמש. במקרה שלנו ה GUI בצד PC.

ביצועים בפועל לעומת מפרט טכני

- מספר הצעדים המשלימים סיבוב שלם של מנוע הצעד ובהתאם אליו נעשו כל החישובים בפרויקט הינו 510 צעדים.
מתוך כך מחושבת זווית צעד במערכת שלנו והיא 0.708 מעלות במקום 0.08 כמפורט בהגדרת המשימה.

אין עוד שינויים שבוצעו בהתאם למפרט הטכני ולהגדרת המשימה.

מעגל האלקטרוני של המערכת



מסקנות והצעות לשיפורים

- התאמת מהירות תנועת הצייר/ המנוע בהתאם לjoystick, כלומר ככל שהjoystick נמצא יותר בקצה כך המנוע/צייר ינועו מהר יותר.
לשם כך נדרש להשתמש בjoystick עם פחות רעשים ולחלק את מרחב המתחים של הג'ויסטיק לחלקים עדינים יותר.

- הוספת פיצ'רים בצד PC עיצוב, צבעים בציור סגנונות ציור שונים כמו ספריי וכו'

- ישנו טרייד-אוף בין מהירות תנועת הצייר לדיוק הציור והיכולת שלו ליצור תנועות מעגליות שניתן לשנותו בקוד בקלות ע"י שינוי פרמטר (delta), עבור joystick יציב יותר וללא רעשים + ערוץ תקשורת מהיר יותר אפשר להגיע לרמת ציור גבוהה ומהירה.